

Road Damage Classification

Experiment Report

Generated: 2026-06-11 15:46:23

1. Dataset

Class	Samples	Ratio
000_damage (damaged)	167	42.7%
001_no_damage (no damage)	224	57.3%
Total	391	100%

- Image size: 256×256 (RGB) or 224×224 (ResNet)
- Split: Train 60% / Validation 20% / Test 20% (random_seed=1)
- Note class imbalance: no_damage is ~1.3x more frequent than damage. Accuracy alone may be misleading — a trivial majority-class predictor can score high.

2. Image Augmentation

The following augmentations were applied to training data to reduce overfitting.

Technique	Effect
Horizontal Flip	Learns position invariance through left-right flipping
Vertical Flip	Vertical inversion; rare in road images but aids generalization
Width/Height Shift ($\pm 10\%$)	Robustness to small positional shifts
Zoom ($\pm 10\%$)	Handles scale variation
Rotation ($\pm 10^\circ$)	Robustness to slight tilts

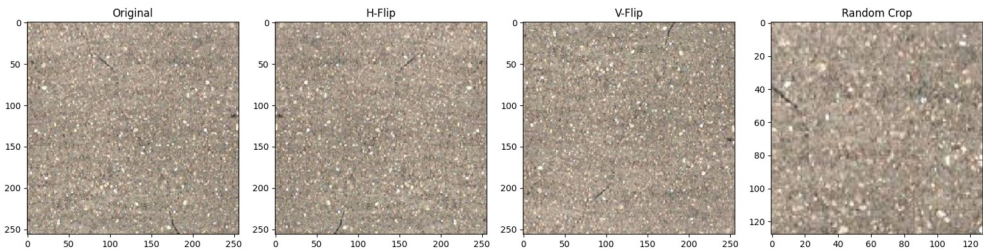


Figure 1: Augmentation examples (Original / H-Flip / V-Flip / Crop)

3. Model Architecture

Model 1: Simple Neural Network

- Input: 256×256 grayscale → Flatten (65,536 dims)
- Dense(512, ReLU) → Dropout(0.2) → Dense(512, ReLU) → Dropout(0.2) → Dense(2, Softmax)
- Optimizer: Adam(lr=0.01) / Epochs: 200 / Batch: 128
- Note: simplest fully-connected network; spatial information is entirely lost.

Model 2: CNN

- Input: 256×256 RGB
- BatchNorm → Conv2D(64, 7×7) → MaxPool → Dropout(0.4)
- BatchNorm → Conv2D(128, 3×3) → MaxPool → Dropout(0.4)
- BatchNorm → Conv2D(256, 5×5) → MaxPool → Dropout(0.4)
- Flatten → Dense(256, ELU) → Dropout(0.5) → Dense(2, Softmax)
- Optimizer: AdamW(lr=0.0001) / Epochs: 5 / Batch: 8
- Note: hierarchically extracts spatial features (edges, textures).

Model 3: ResNet50 (Transfer Learning)

- Input: 224×224 RGB
- ResNet50 (ImageNet pre-trained, include_top=True)
- Flatten → Dense(256, ELU) → Dropout(0.5) → Dense(2, Softmax)
- Optimizer: AdamW(lr=0.0001) / Epochs: 5 / Batch: 8
- Note: 25M+ parameter deep network. Transfer learning enables rich feature representations even on small datasets.

4. Results per Model

4-1. Simple Neural Network

Hyperparameters: Epochs=200, Batch=128, Optimizer=Adam(lr=0.01)

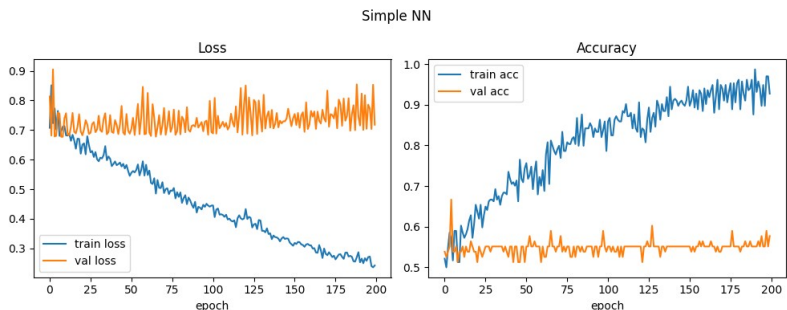


Figure 2: Simple NN training curves

Test Results

Metric	Value
--------	-------

Test Loss	0.7439
Test Accuracy	60.8%
TP — correctly detected damage	3
TN — correctly excluded no_damage	45
FN — missed damage	30
FP — false alarm (no_damage predicted as damage)	1
Total test images	79

Confusion Matrix

	Predicted: damage	Predicted: no_damage
Actual: damage	3 (TP ✓)	30 (FN x missed)
Actual: no_damage	1 (FP x false alarm)	45 (TN ✓)

Classification Report

Class	Precision	Recall	F1-score	Support
000_damage	0.750	0.091	0.162	33
001_no_damage	0.600	0.978	0.744	46
weighted avg	0.663	0.608	0.501	79

Comments

- Accuracy 60.8% — must be interpreted in light of class imbalance (no_damage majority).
- **damage Recall = 0.091 (9.1%) — share of actual damage cases detected. Most critical metric for road management. FN=30 missed cases.**
- damage Precision = 0.750 — of all damage predictions, the proportion that were truly damaged. FP=1 false alarm.
- damage F1-score = 0.162 — harmonic mean of Precision and Recall; overall metric under class imbalance.

4-2. CNN

Hyperparameters: Epochs=5, Batch=8, Optimizer=AdamW(lr=0.0001)

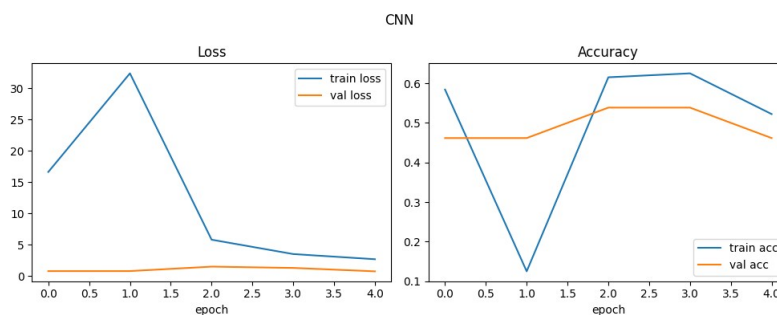


Figure 3: CNN training curves

Test Results

Metric	Value
Test Loss	0.7411
Test Accuracy	41.8%
TP — correctly detected damage	33
TN — correctly excluded no_damage	0
FN — missed damage	0 ★ best
FP — false alarm (no_damage predicted as damage)	46
Total test images	79

Confusion Matrix

	Predicted: damage	Predicted: no_damage
Actual: damage	33 (TP ✓)	0 (FN x missed)
Actual: no_damage	46 (FP x false alarm)	0 (TN ✓)

Classification Report

Class	Precision	Recall	F1-score	Support
000_damage	0.418	1.000	0.589	33
001_no_damage	0.000	0.000	0.000	46
weighted avg	0.174	0.418	0.246	79

Comments

- Accuracy 41.8% — extreme bias: all samples predicted as damage.
- damage Recall = 1.000 (100%) ★ — zero missed damage cases; safest from a road management perspective.
- damage Precision = 0.418 — high false-alarm rate (FP=46).
- damage F1-score = 0.589

4-3. ResNet50 (Transfer Learning)

Hyperparameters: Epochs=5, Batch=8, Optimizer=AdamW(lr=0.0001)

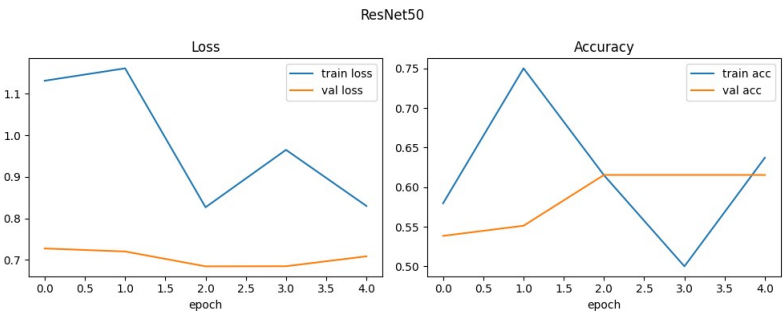


Figure 4: ResNet50 training curves

Test Results

Metric	Value
Test Loss	0.5606 ★ lowest
Test Accuracy	72.2% ★ highest
TP — correctly detected damage	26
TN — correctly excluded no_damage	31
FN — missed damage	7
FP — false alarm (no_damage predicted as damage)	15
Total test images	79

Confusion Matrix

	Predicted: damage	Predicted: no_damage
Actual: damage	26 (TP ✓)	7 (FN x missed)
Actual: no_damage	15 (FP x false alarm)	31 (TN ✓)

Classification Report

Class	Precision	Recall	F1-score	Support
000_damage	0.634	0.788	0.703	33
001_no_damage	0.816	0.674	0.738	46
weighted avg	0.740	0.722	0.723	79

Comments

- Accuracy 72.2% ★ — highest among all three models.
- damage Recall = 0.788 (78.8%) — substantially fewer missed cases (FN=7).
- damage F1-score = 0.703 ★ — best balance of Precision and Recall.

5. Model Comparison

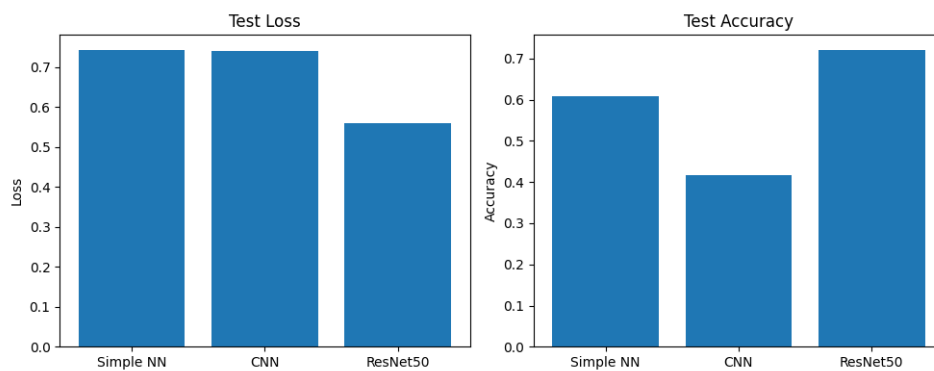


Figure 5: Loss & Accuracy comparison across all 3 models

Model	Test Loss	Test Accuracy	damage F1	damage Recall	FN missed	FP false alarm
Simple NN	0.7439	60.8%	0.162	0.091	30	1
CNN	0.7411	41.8%	0.589	1.000 ★	0 ★	46
ResNet50	0.5606 ★	72.2% ★	0.703 ★	0.788	7	15

★ = best value in each column

Discussion

In road damage detection, the cost of missed detections (FN) is highest — undetected damage poses an accident risk. Therefore, damage class Recall should be prioritized over Accuracy when selecting the final model.

Summary of model characteristics:

- Simple NN — structurally disadvantaged for image classification since spatial information is discarded. Extremely low Recall leads to many missed damage cases.
- CNN — detects all damage (Recall=100%) but also flags every no_damage sample as a false alarm. Epoch count and regularization need tuning.
- **ResNet50 — best Accuracy and F1. Transfer learning from ImageNet leverages rich feature representations even with a small dataset (391 images), making it the recommended choice.**

Homework

Question 1 (20 points)

You are a road administrator in a local government. A systems engineering company says:

“We have developed a road inspection software with 99% accuracy! Please purchase it!”

What should you be skeptical about?

Answer:

Suspect artificially high accuracy caused by class imbalance.

In road inspection datasets, the number of normal road images is usually much larger than the number of damaged road images. Even in this dataset, there is already an imbalance (167 damaged images vs. 224 normal images). In real-world road networks, damaged sections are even rarer.

For example, if damaged roads account for only 1% of the data and normal roads account for 99%, a model that always predicts “normal” would still achieve 99% accuracy.

The confusion matrix from the exercise showed cases where the Recall for the damaged class was 0. The ResNet50 model used in the exercise was an example of this situation.

Metrics that should be checked

Metric	Reason
Recall	Measures whether actual damaged roads are being missed. This is the most important metric.
Precision	Measures how many predicted damages are actually damaged.
F1 Score	Harmonic mean of Precision and Recall. A reliable overall metric for imbalanced datasets.
Confusion Matrix	Allows direct inspection of false negatives (missed damages).

In road management, the cost of false negatives (FN) is extremely high because missing a damaged road can lead to safety risks. Therefore, Recall should be prioritized over Accuracy when evaluating the model.

Question 2 (20 points)

Computers do not get tired and can continue training a model indefinitely. However, increasing the number of epochs without limit is not practical. Explain why.

Answer:

Because overfitting occurs.

As the number of training epochs increases, the model begins to memorize noise and outliers in the training data.

As a result, the training loss continues to decrease, while the validation or test loss eventually starts increasing.

Other reasons

- Training time and computational cost increase with the number of epochs.
- If the learning rate remains fixed, the loss may oscillate around a minimum and stop improving after convergence.

Common countermeasures

- Early Stopping (automatically stops training when validation performance no longer improves)
- ReduceLROnPlateau (reduces the learning rate when improvement stagnates)
- Dropout and Batch Normalization (help reduce overfitting)

Question 3 (30 points)

Modify the learning rate, other hyperparameters, or the neural network architecture. Compare the results with those of the sample model. (You may attach screenshots of the model configuration and training results.)

Answer:

Examples of parameter changes and their effects

Changing the learning rate

Learning Rate	Behavior
Too large (0.1)	Loss oscillates or diverges; training becomes unstable
Appropriate (0.0001)	Stable convergence
Too small (0.000001)	Convergence is very slow and may require many more epochs

Changing the dropout rate

Dropout Rate	Behavior
0 (none)	High training accuracy but lower validation accuracy (overfitting)
0.4–0.5	Better generalization performance (sample configuration)
0.8 (too high)	Model struggles to learn; both training and validation accuracy decrease

Increasing the number of epochs (5 → 20)

- When the dataset is small, the risk of overfitting increases.
- Although Dropout and Batch Normalization help mitigate overfitting, validation loss should still be monitored carefully.

Question 4 (30 points)

Suppose the input image size is changed from 256×256 to 512×512 . How does the number of parameters in each layer change?

Also, suppose all convolution kernels are changed from 5×5 to 3×3 . How does the number of parameters in each layer change?

Show the detailed calculations. (You may base your answer on the CNN model used in the Colab lecture materials.)

Answer

Consider a custom CNN with three convolutional layers.

Assumptions:

- Input size: 256×256
- Grayscale image (1 channel)
- padding= 'same '

The number of parameters in a Conv2D layer is calculated as:

$(\text{kernel height} \times \text{kernel width} \times \text{input channels} + 1) \times \text{number of filters}$

Original parameter counts

Layer	Calculation	Parameters
Conv2D(64, 7×7)	$(7 \times 7 \times 1 + 1) \times 64$	3,200
Conv2D(128, 3×3)	$(3 \times 3 \times 64 + 1) \times 128$	73,856
Conv2D(256, 5×5)	$(5 \times 5 \times 128 + 1) \times 256$	819,456
Flatten $\rightarrow$ Dense(256)	$(32 \times 32 \times 256 + 1) \times 256$	67,109,120
Dense(2)	$(256 + 1) \times 2$	514

Flatten size calculation

Since padding= 'same ' preserves spatial dimensions:

$256 \times 256 \rightarrow \text{MaxPool}(2, 2) \rightarrow 128 \times 128$
 $\rightarrow \text{MaxPool}(2, 2) \rightarrow 64 \times 64$
 $\rightarrow \text{MaxPool}(2, 2) \rightarrow 32 \times 32 \times 256$

Flatten $\rightarrow 32 \times 32 \times 256 = 262,144$

4a. Changing the input size from 256×256 to 512×512

The number of parameters in convolutional layers depends only on:

- kernel size
- input channels
- number of filters

Therefore, the Conv2D layers are unchanged.

However, the Flatten output becomes four times larger, causing the Dense(256) layer to have four times as many parameters.

$512 \times 512 \rightarrow \text{MaxPool} \rightarrow 256 \times 256$
 $\rightarrow \text{MaxPool} \rightarrow 128 \times 128$
 $\rightarrow \text{MaxPool} \rightarrow 64 \times 64 \times 256$

Flatten $\rightarrow 64 \times 64 \times 256 = 1,048,576$

Layer	256×256 Input	512×512 Input	Change
Conv2D(64, 7×7)	3,200	3,200	No change
Conv2D(128, 3×3)	73,856	73,856	No change
Conv2D(256, 5×5)	819,456	819,456	No change
Dense(256)	67,109,120	268,435,712	Approximately 4× larger
Dense(2)	514	514	No change

4b. Changing the 5×5 kernel to a 3×3 kernel

Only the third convolutional layer changes.

Because padding= 'same' is used, the spatial dimensions remain unchanged, so the Dense layers are unaffected.

Before:

$$(5 \times 5 \times 128 + 1) \times 256 \\ = 819,456$$

After:

$$(3 \times 3 \times 128 + 1) \times 256 \\ = 295,168$$

Reduction:

$$819,456 - 295,168 \\ = 524,288 \text{ fewer parameters}$$

Layer	5×5 Kernel	3×3 Kernel	Change
Conv2D(256)	819,456	295,168	Reduced to about 36% of the original
Dense(256)	67,109,120	67,109,120	No change

Summary

- Changing the input image size does not affect the number of parameters in convolutional layers, but it significantly affects the number of parameters in fully connected (Dense) layers.
- Changing the kernel size affects the number of parameters in convolutional layers but does not affect the Dense layers when the output feature-map dimensions remain unchanged.